Abstract Writing For Research

Optimization Problem in Research

A good heuristic is:

- **Optimize for Novelty/Innovation (= contribution)**
- **Maximize surprise ~ Bit Flip!**
- **Teach the world some new**

Abstract writing with Intro technique we taught

Same as the intro (for the abstract), except the intro dedicated an entire paragraph to the 6 bullet points you have, abstract does 1-2 sentences instead:

1. Problem motivation: Explain the main problem and why it matters, optionally why it's not been solved.

2. Set up the bit/assumption: Identify the assumption in prior work you're challenging/inverting.

3. Flip the bit (=contribution): Introduce your novel idea/contribution that challenges/inverts the assumption of prior work.

4. Instantiate the bit flip (=contribution/your solution): Describe how your solution concretely applies the new idea.

5. Evaluation: Briefly mention your method for evaluating the solution.

6. Implications: Discuss the broader impact and significance of your findings.

Abstract Writing

Important, since **Title** + **Abstract**~Summary might be all people read

1. Problem motivation: State problem/Intro + Mini background/motivation (want to people to want to continue read)

2. Bit flip: Old Assumption → New Idea

3. Flip the bit (=contribution): Introduce your novel idea/contribution that challenges/inverts the assumption of prior work. Your approach/solution. Factual, comprehensive, extremely succinct/direct

4. Instantiate the bit flip (=contribution/your solution): Describe how your solution concretely applies the new idea.

5. Evaluation: Briefly mention your method for evaluating the solution.

6. Broader Perspective/ Implications

Craft carefully & re-write every sentence many times

Abstract Writing

Important: Title + Abstract/Sum.

(Might be all they read!)

1. State Problem/Intro
2. Mini Motivation/Background
3. Bit flip: Old Ass. → New i.
4. Your approach/solution

Factual, succinct/direct

5. Broader Perspective/ Implications

Abstract

Recent work claims that large language models display *emergent abilities*: abilities not present in smaller-scale models that are present in larger-scale models. What

not present in smaller-scale models that are present in larger-scale models. What makes emergent abilities intriguing is two-fold: their *sharpness*, transitioning seemingly instantaneously from not present to present, and their *unpredictability*, appearing at seemingly unforeseeable model scales. Here, we present an alternative

appearing at seemingly unforeseeable model scales. Here, we present an alternative explanation for emergent abilities: for a particular task and model family, when analyzing fixed model outputs, emergent abilities appear due to the researcher's choice of metric rather than due to fundamental changes in models with scale. Specifically, nonlinear or discontinuous metrics produce seemingly emergent abilities, whereas linear or continuous metrics produce smooth, continuous, predictable changes in model performance. We present our alternative explanation in a simple mathemati-

appearing at seemingly unforeseeable model scales. Here, we present an alternative explanation for emergent abilities: for a particular task and model family, when analyzing fixed model outputs, emergent abilities appear due to the researcher's choice of metric rather than due to fundamental changes in models with scale. Specifically, nonlinear or discontinuous metrics produce seemingly emergent abilities, whereas linear or continuous metrics produce smooth, continuous, predictable changes in model performance. We present our alternative explanation in a simple mathemati-

across diverse deep network architectures. Via all three analyses, we provide evidence that emergent abilities disappear with different metrics or with better statistics, and may not be a fundamental property of scaling AI models.

How to construct a *Nature* summary paragraph

Annotated example taken from *Nature* 435, 114–118 (5 May 2005).

One or two sentences providing a **basic introduction** to the field, comprehensible to a scientist in any discipline.

Two to three sentences of **more detailed background**, comprehensible to scientists in related disciplines.

One sentence clearly stating the **general problem** being addressed by this particular study.

One sentence summarizing the main result (with the words "**here we show**" or their equivalent).

Two or three sentences explaining what the **main result** reveals in direct comparison to what was thought to be the case previously, or how the main result adds to previous knowledge.

One or two sentences to put the results into a more **general context**.

Two or three sentences to provide a **broader perspective**, readily comprehensible to a scientist in any discipline, may be included in the first paragraph if the editor considers that the accessibility of the paper is significantly enhanced by their inclusion. Under these circumstances, the length of the paragraph can be up to 300 words. (This example is 190 words without the final section, and 250 words with it).

During cell division, mitotic spindles are assembled by microtubule-based motor proteins[1,2]. The bipolar organization of spindles is essential for proper segregation of chromosomes, and requires plus-end-directed homotetrameric motor proteins of the widely conserved kinesin-5 (BimC) family[3]. Hypotheses for bipolar spindle formation include the 'push–pull mitotic muscle' model, in which kinesin-5 and opposing motor proteins act between overlapping microtubules[2,4,5]. However, the precise roles of kinesin-5 during this process are unknown. Here we show that the vertebrate kinesin-5 Eg5 drives the sliding of microtubules depending on their relative orientation. We found in controlled *in vitro* assays that Eg5 has the remarkable capability of simultaneously moving at ~20 nm s^{-1} towards the plus-ends of each of the two microtubules it crosslinks. For anti-parallel microtubules, this results in relative sliding at ~40 nm s^{-1}, comparable to spindle pole separation rates *in vivo*[6]. Furthermore, we found that Eg5 can tether microtubule plus-ends, suggesting an additional microtubule-binding mode for Eg5. Our results demonstrate how members of the kinesin-5 family are likely to function in mitosis, pushing apart interpolar microtubules as well as recruiting microtubules into bundles that are subsequently polarized by relative sliding. We anticipate our assay to be a starting point for more sophisticated *in vitro* models of mitotic spindles. For example, the individual and combined action of multiple mitotic motors could be tested, including minus-end-directed motors opposing Eg5 motility. Furthermore, Eg5 inhibition is a major target of anti-cancer drug development, and a well-defined and quantitative assay for motor function will be relevant for such developments.

https://www.nature.com/documents/nature-summary-paragraph.pdf

Abstract serve to Tell Reader if to Read Your paper

	Old (Brando)	New (Sanmi)
Opening	Taxonomy of what benchmarks do wrong (3 sentences)	Names the failure mode in 1 sentence: "semantically disconnected"
Hook	Line 7 before reader knows what's new	Line 1 — immediate
Metric	Not mentioned until body	"Multiplicative evaluation metric" in abstract
Finding	Placeholder: "[key finding]"	Concrete: "X% → Y% drop"
Memorable term	None	"Verification hallucinations"
Closing	"For the first time… genuinely trustworthy" (self-congratulatory)	"From isolated theorem proving to trustworthy verification" (vision)
Weakness	Too detailed for 30-second scan	Drops data quality / no-LLM-judge point

Too detailed vs Punchier/Direct

Abstract

Current autoformalization benchmarks either verify the formal model in isolation, perform one-way translations without validating against the original specifications, or ignore the source code entirely—in all cases, no rigorous connection is established between the original program and its formal counterpart. This reflects a pervasive assumption that end-to-end verification means proving properties of the formal model alone, yet without grounding back to the original code, verification guarantees do not transfer to the software that developers actually write and deploy. We introduce VERIBENCH-DT, a benchmark and evaluation framework that closes this gap by requiring agentic autoformalization to be validated end-to-end—connecting original Python programs to their Lean 4 formalizations through differential testing, where both implementations are executed against shared input–output test suites to verify behavioral equivalence. Concretely, we contribute a curated dataset of Python–Lean 4 program pairs, a novel metric grounded in differential testing, and an evaluation protocol that measures whether AI agents can produce formal models whose behavior provably aligns with the original code. Our benchmark ensures data quality through state-of-the-art AI agentic generation paired with rigorous human curation with a high quality rubric, and our metric relies on trustworthy symbolic test execution rather than LLM-based judges. We evaluate N state-of-the-art agents and find [key finding—e.g., that even the best agents fail to achieve end-to-end alignment on $X\%$ of tasks]. VERIBENCH-DT enables the AI research community to assess, for the first time, whether today's AI agents can deliver genuinely trustworthy code verification for programs written in languages developers actually use.

Abstract

Current autoformalization systems for code often successfully prove theorems
are semantically disconnected from the source code they intend to verify.
introduce VERIBENCH-DT, a benchmark that enforces end-to-end grounding
validating agent-generated Lean 4 formalizations against original Python progra
via differential testing. Unlike existing benchmarks that rely on static translat
checks or LLM-based judges, VERIBENCH-DT executes both the source (Pyth
and the target (Lean) on shared inputs to verify behavioral equivalence.
contribute a curated dataset of N Python–Lean pairs and a novel multiplica
evaluation metric that exposes *verification hallucinations*—cases where age
produce correct proofs for incorrect specifications. Evaluation of state-of-the
agents reveals a critical gap: while models achieve $X\%$ success on formal pr
generation, their end-to-end reliability drops to $Y\%$ when grounded against orig
code execution. VERIBENCH-DT provides the necessary tooling to meas
and close this gap, moving autoformalization from isolated theorem proving
trustworthy software verification.

DRAFT 1 (The 'Complete' Intro)



Opening
Taxonomy of what benchmarks do wrong (3 sentences).



The Hook
Line 7 before reader knows what's new.



Metric
Not mentioned until body.



Vocabulary
None.



Finding
Placeholder: '[key finding]'.



Closing
For the first time... genuinely trustworthy...'

DRAFT 2 (The 'Billboard')



Opening
Names the failure mode in 1 sentence: **'semantically disconnected'**.



The Hook
Line 1—immediate.



Metric
Multiplicative evaluation metric' in abstract.



Vocabulary
'Verification hallucinations'.



Finding
Concrete: **'X% → Y% drop'**.



Closing
'From isolated theorem proving to **trustworthy verification**'.